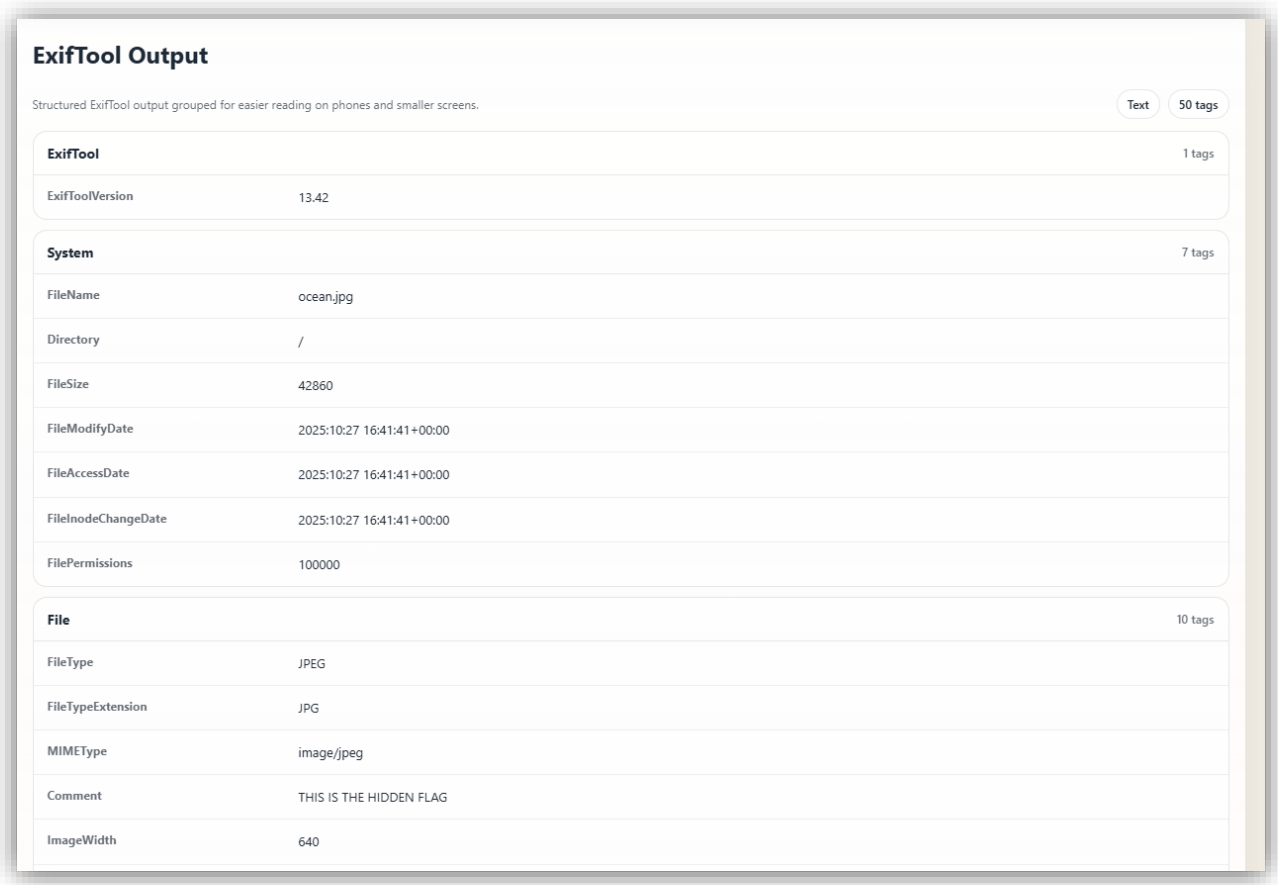
WEEK 4 : Metadata and File Analysis Report

1. Ocean.jpg



ExifTool Output

Structured ExifTool output grouped for easier reading on phones and smaller screens.

Text | 50 tags

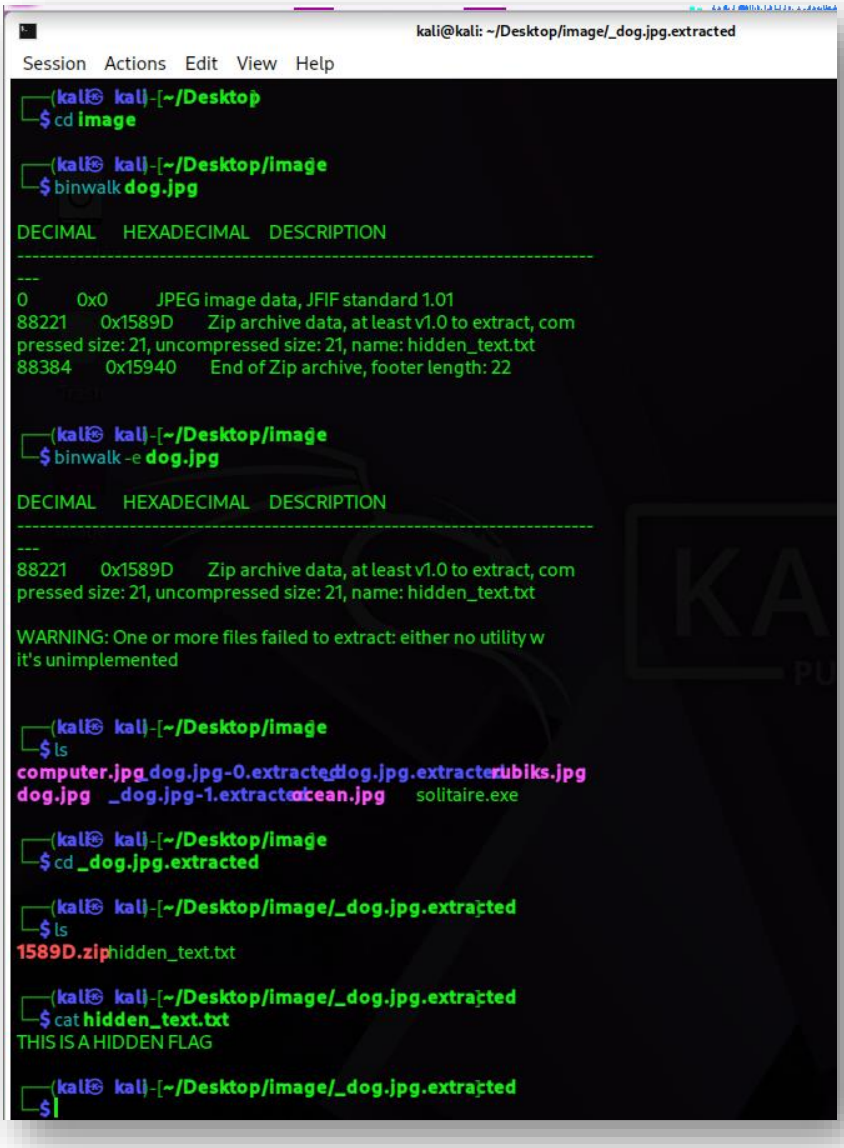
ExifTool	1 tags
ExifToolVersion	13.42

System	7 tags
FileName	ocean.jpg
Directory	/
FileSize	42860
FileModifyDate	2025:10:27 16:41:41+00:00
FileAccessDate	2025:10:27 16:41:41+00:00
FileInodeChangeDate	2025:10:27 16:41:41+00:00
FilePermissions	100000

File	10 tags
FileType	JPEG
FileTypeExtension	JPG
MIMEType	image/jpeg
Comment	THIS IS THE HIDDEN FLAG
ImageWidth	640

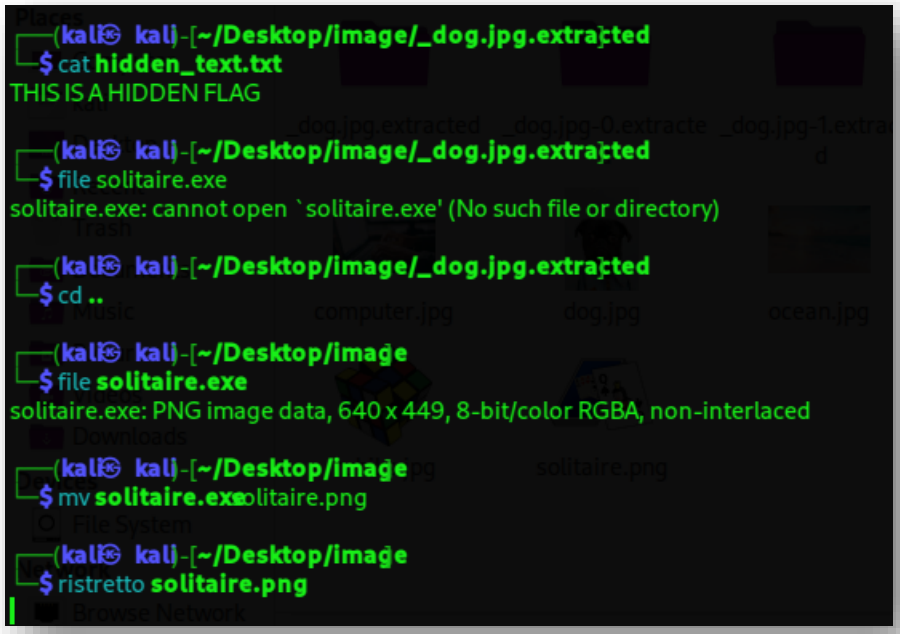
I analyzed ocean.jpg to see if any flags were hidden within its background data. I used an online tool called exif.tools to browse the file's metadata. By inspecting the output, I found a specific field labeled "Comment". This section explicitly contained the string "THIS IS THE HIDDEN FLAG," which served as the first discovery.

2. dog.jpg

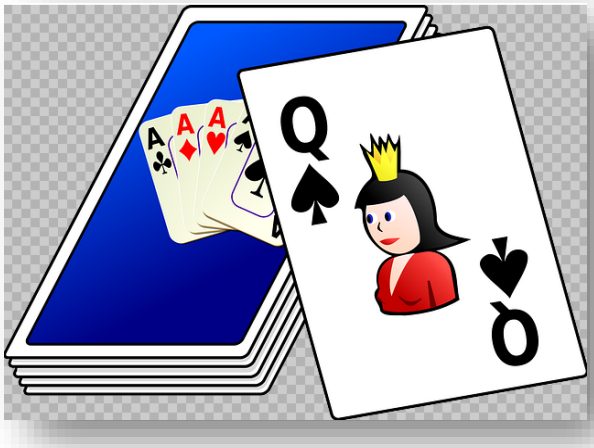


The next task involved dog.jpg, which I analyzed using the Linux command line on a Kali Linux machine. First, I ran the binwalk command to scan the image, which revealed a hidden Zip archive and a file named hidden_text.txt embedded within the image data. To access this, I used binwalk -e to extract the hidden files into a new directory named _dog.jpg.extracted. After navigating into that folder and using the ls command to confirm the files, I used cat hidden_text.txt to read the contents. This successfully revealed the hidden flag: "THIS IS A HIDDEN FLAG".

3. Solitaire



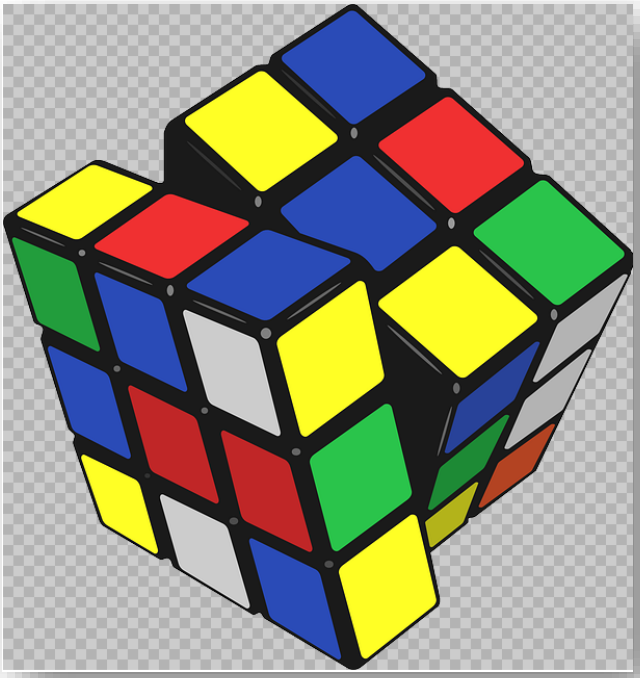
For the third file, solitaire.exe, the goal was to verify if the file was actually an executable. I used the file command in the terminal to check its true signature. The system reported that the file was actually PNG image data with a resolution of 640 x 449, proving the .exe extension was a decoy. To fix this, I used the mv command to rename it to solitaire.png. Finally, I opened the file using the ristretto image viewer to see the actual image of playing cards.



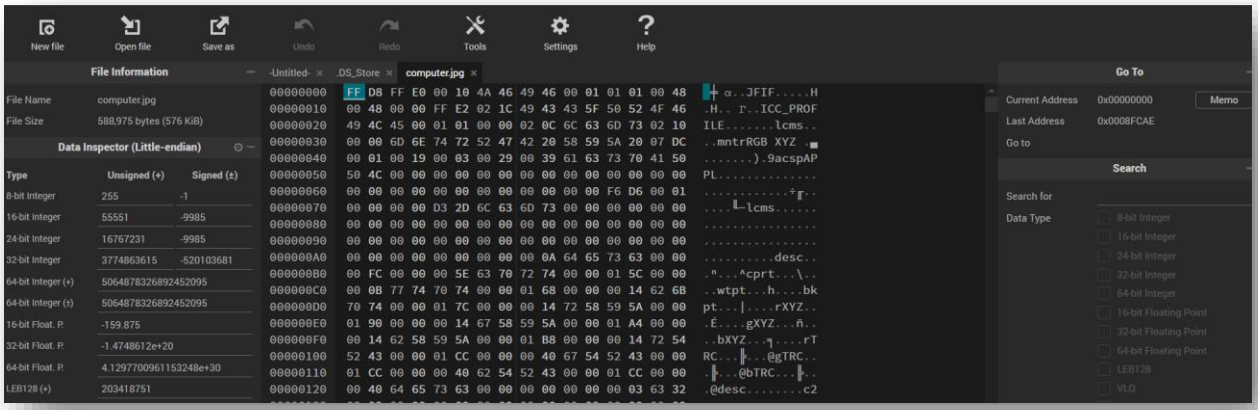
4. Rubiks

```
┌──(kali㉿ kali)-[~/Desktop/image
└─$ file rubiks.jpg
rubiks.jpg: PNG image data, 609 x 640, 8-bit/color RGBA, non-interlaced

┌──(kali㉿ kali)-[~/Desktop/image
└─$ mv rubriks.jpg rubriks.png
mv: cannot stat 'rubriks.jpg': No such file or directory

┌──(kali㉿ kali)-[~/Desktop/image
└─$ mv rubiks.jpgubiks.png

┌──(kali㉿ kali)-[~/Desktop/image
└─$ ristretto rubiks.png
```

The analysis of rubiks.jpg followed a similar process to identify another extension mismatch. When I ran the file command on it, the output confirmed that it was also PNG image data rather than a JPEG. I used the mv command to change the extension to .png to match its true format. Once the extension was corrected, I was able to open it with the ristretto tool, which displayed a 3D graphic of a Rubik's Cube.



5. computer.png



The final part of the lab involved computer.jpg, where I needed to inspect the raw binary data. I first uploaded the file to hexed.it, a web-based hex editor, to view the file headers and hexadecimal values.

To make the data more readable, I also used the dCode Strings Extractor tool. By processing the file through this tool, I was able to pull out human-readable text strings from the binary code, such as ICC_PROFILE and mntrRGB XYZ. This demonstrated how a combination of hex editing and string extraction can be used to analyze a file's internal structure.